

Giải Thích Toàn Bộ Script Trong Assets/Scripts

Tài liệu này giải thích chi tiết vai trò của từng script trong project Unity hiện tại, cách các script phối hợp với nhau, luồng xử lý chính của gameplay, cùng các điểm mạnh và giới hạn kỹ thuật đang có.

Phạm vi tài liệu: AspectRatioHandler.cs, Block.cs, Blocks.cs, Board.cs, Cell.cs, GameOverManager.cs, Polyominos.cs, ScoreManager.cs.

1. Tổng quan kiến trúc

- Board là lõi gameplay: quản lý ma trận 8x8, kiểm tra đặt khối, tô preview, xác định hàng/cột đầy và xóa line.
- Blocks là bộ điều phối các khối đang chờ người chơi kéo thả.
- Block là logic của từng khối riêng lẻ: hiển thị polyomino, xử lý drag/drop và giao tiếp với Board.
- Cell là đơn vị hiển thị nhỏ nhất của cả board và block preview.
- Polyominos là kho dữ liệu shape.
- ScoreManager quản lý điểm hiện tại và high score.
- GameOverManager theo dõi điều kiện thua và điều khiển UI game over + restart.
- AspectRatioHandler giữ tỉ lệ 9:16 ổn định trên WebGL / nhiều độ phân giải.

Luồng tổng quát: Blocks sinh ra 3 khối khả dụng, người chơi kéo một Block, script đó hỏi Board xem vị trí thả có hợp lệ hay không. Nếu hợp lệ, Board cập nhật dữ liệu ma trận, xóa line nếu cần và gọi ScoreManager cộng điểm. Sau mỗi lần dùng khối, Blocks kiểm tra còn nước đi hay không. Nếu hết nước đi, GameOverManager bật overlay Game Over.

2. Giải thích từng script

2.1. Board.cs

Mục đích chính: Đây là script quan trọng nhất của gameplay. Nó đại diện cho bàn chơi 8x8 và chịu trách nhiệm giữ trạng thái toàn bộ ô trên board.

- Dữ liệu cốt lõi: dùng mảng 2 chiều `Cell[,]` để tham chiếu tới object hiển thị và mảng `int[,]` tên data để lưu trạng thái logic của từng ô. Quy ước hiện tại là 0 = rỗng, 1 = hover preview, 2 = đã chiếm.
- Khởi tạo: trong `Awake()` script đảm bảo luôn có transform cha tên `Cells`; trong `Start()` nó instantiate 64 cell từ prefab rồi ẩn đi.
- Hover / preview: phương thức `Hover()` nhận điểm đặt và index của polyomino, tính ra các ô mà shape sẽ chiếm, kiểm tra từng ô bằng `IsValidPoint()`, sau đó hiển thị preview bán trong suốt lên board.

- Highlight line sắp hoàn thành: script có hai nhóm hàm PredictFullLineColumns/Rows() và HighlightFullLineColumns/Rows() để tô sáng các line sẽ được xóa nếu người chơi thả khối ở vị trí đang hover.
- Đặt khối: Place() chuyển trạng thái các ô hover từ 1 sang 2, gọi hàm xóa line, rồi dọn dữ liệu tạm.
- Xóa line: ClearFullLines() chỉ quét các hàng/cột nằm trong footprint của shape vừa đặt, không quét toàn board một cách mù quáng. Nếu hàng/cột đầy, script reset ô về 0 và ẩn cell tương ứng.
- Kiểm tra còn nước đi: CheckPlace() duyệt toàn bộ vị trí hợp lệ trên board cho một shape bất kỳ để xác định có còn chỗ đặt hay không.

Vai trò trong hệ thống: Board là “single source of truth” cho trạng thái bàn chơi. Mọi logic drag/drop, clear line, tính nước đi còn lại, highlight preview đều đi qua script này.

2.2. Block.cs

Mục đích chính: đại diện cho một khối kéo thả cụ thể trong hàng chờ của người chơi.

- Khởi tạo cell con: trong Initialize(), mỗi Block tạo sẵn ma trận 5x5 cell con từ prefab. Đây là dạng pre-allocation để tái sử dụng, thay vì mỗi lần đổi shape lại instantiate mới.
- Hiển thị shape: Show() lấy dữ liệu shape từ Polyominos, tính tâm shape rồi đặt localPosition cho các cell con đang được dùng.
- Tương tác chuột: OnMouseDown(), OnMouseDown(), OnMouseDown() xử lý toàn bộ vòng đời kéo-thả.
- Preview khi kéo: khi người chơi kéo block, script liên tục quy đổi vị trí chuột sang tọa độ board, gọi Board.Hover() để cập nhật preview trên bàn.
- Đặt thành công: nếu Board.Place() trả về true, block sẽ bị ẩn khỏi hàng chờ bằng SetActive(false) và báo về Blocks.Remove().
- Thứ tự hiển thị: dùng SortingGroup để block đang kéo nổi lên trên các block khác.

Vai trò trong hệ thống: Block là lớp trung gian giữa input của người chơi và Board. Nó không tự quyết định luật chơi; nó chỉ gửi yêu cầu tới Board và phản ánh kết quả ra UI/visual.

2.3. Blocks.cs

Mục đích chính: quản lý cả cụm 3 block khả dụng cho người chơi ở mỗi lượt.

- Bố trí vị trí block: trong Start(), script tính kích thước hiển thị và vị trí ngang của từng block dựa trên kích thước board.
- Khởi tạo shape: tạo mảng polyominoIndexes để lưu shape hiện tại của từng block.
- Sinh bộ block mới: Generate() random shape cho toàn bộ block, bật lại gameObject và gọi Show().
- Đếm số block còn dùng được: biến blockCount giảm dần mỗi khi người chơi dùng xong một khối; khi về 0, script generate lại batch mới.
- Kiểm tra thua: trong Remove(), script duyệt từng block còn active và gọi board.CheckPlace(). Nếu không còn block nào có thể đặt, biến local lose sẽ giữ giá trị true.

Vai trò trong hệ thống: script này giống một “wave manager” cho block. Nó không xử lý drag/drop chi tiết, mà phụ trách phát block, reset block và quyết định logic còn nước đi / hết nước đi.

2.4. Cell.cs

Mục đích chính: điều khiển visual của một ô vuông đơn lẻ.

- Giữ 2 sprite: normal và highlight.
- 3 trạng thái hiển thị chính: Normal() cho ô đã chiếm bình thường, Highlight() cho ô được tô sáng, và Hover() cho preview bán trong suốt.
- Tối ưu tái sử dụng: Hide() chỉ tắt object, không destroy.

Vai trò trong hệ thống: Cell là lớp hiển thị thuần visual. Nó không giữ luật chơi, chỉ nhận lệnh từ Board hoặc Block để đổi sprite / màu / active state.

2.5. Polyominos.cs

Mục đích chính: lưu toàn bộ dữ liệu shape của game dưới dạng ma trận nhị phân.

- Dữ liệu tĩnh: mỗi shape được lưu dưới dạng int[,] với giá trị 1 là có ô và 0 là rỗng.
- Khởi tạo một lần: static constructor chạy ngay lần đầu class được dùng, đảo ngược thứ tự row bằng ReverseRows() để phù hợp với hệ tọa độ hiển thị của game.
- Truy cập shape: Get(index) trả shape theo index; Length cho biết tổng số shape đang có.

Vai trò trong hệ thống: đây là “data source” cho các shape. Nhờ tách riêng dữ liệu shape khỏi logic kéo thả và board, code dễ mở rộng thêm pattern mới hơn.

2.6. ScoreManager.cs

Mục đích chính: quản lý điểm hiện tại và high score.

- Singleton: dùng thuộc tính tĩnh Instance để các script khác, cụ thể là Board, có thể gọi cộng điểm một cách trực tiếp.
- Persistence: trong Awake(), script nạp high score từ PlayerPrefs bằng key HighScore.
- Cộng điểm: AddScore(int points) tăng điểm hiện tại. Nếu vượt high score cũ, script lưu lại ngay bằng PlayerPrefs.SetInt() và PlayerPrefs.Save().
- Cập nhật UI: RefreshUI() đẩy giá trị mới ra TMP text của current score và high score.

Vai trò trong hệ thống: script này là điểm tập trung cho logic score. Board chỉ cần biết “bao nhiêu line vừa clear” rồi chuyển thành điểm, còn toàn bộ UI + lưu trữ được gom vào một nơi.

2.7. GameOverManager.cs

Mục đích chính: quản lý trạng thái game over và UI restart.

- Không sửa trực tiếp Blocks.cs: script này dùng reflection để đọc các field private trong Blocks, gồm board, danh sách block, và polyomino index.
- Phát hiện thua: trong Update(), script gọi HasLoseState(). Hàm này lặp qua mọi block còn active và dùng Board.CheckPlace() để xác nhận còn nước đi hay không.

- Hiển thị UI: khi xác định thua, ShowGameOver() bật panel game over, vô hiệu hoá Blocks và tắt input của từng block.
- Restart: nút restart được gắn listener gọi
SceneManager.LoadScene(SceneManager.GetActiveScene().name) để reset toàn bộ scene.

Vai trò trong hệ thống: đây là lớp quản lý “state chuyển pha” của game: từ đang chơi sang game over, rồi từ game over sang restart.

2.8. AspectRatioHandler.cs

Mục đích chính: giữ khung nhìn 9:16 ổn định trên nhiều độ phân giải, đặc biệt là WebGL.

- ExecuteAlways: script chạy cả trong editor và runtime.
- Theo dõi thay đổi kích thước: trong Update(), nếu Screen.width hoặc Screen.height đổi, script tính lại viewport rect của camera.
- Letterbox / pillarbox: nếu màn hình rộng hơn 9:16, script thu hẹp chiều ngang viewport và canh giữa; nếu màn hình cao hơn 9:16, script thu hẹp chiều dọc viewport.
- Màu viền: camera được ép dùng SolidColor và màu nền đen để tạo vùng viền hai bên hoặc trên/dưới.

Vai trò trong hệ thống: script này xử lý phần “presentation layer” ở mức camera để gameplay và UI giữ đúng bố cục portrait dù đang chạy editor, mobile hay browser WebGL.

3. Mối liên hệ giữa các script

- Blocks giữ danh sách các Block.
- Block gọi Board Hover() và Board Place().
- Board gọi ScoreManager.Instance.AddScore() khi có line được xóa.
- GameOverManager quan sát trạng thái logic của Blocks và Board để xác định game over.
- AspectRatioHandler không can thiệp gameplay, chỉ ảnh hưởng cách camera hiển thị game.

4. Điểm mạnh kỹ thuật hiện tại

- Kiến trúc chia rõ data, gameplay logic, UI/state, và presentation.
- Dùng lại object thay vì destroy liên tục cho cell và block preview, giúp giảm chi phí runtime.
- Logic board khá quyết đoán: state bàn cờ được gom vào một mảng 2 chiều nên việc kiểm soát placement / clear line / check lose dễ theo dõi.
- UI và WebGL đã có lớp bảo vệ aspect ratio riêng, phù hợp với game portrait.

5. Một số lưu ý / giới hạn hiện tại

- GameOverManager đang dùng reflection để đọc field private của Blocks. Cách này hoạt động được nhưng không đẹp bằng việc expose API rõ ràng từ script gốc.

- Blocks.cs hiện chỉ log khi thua; phần kích hoạt UI game over được tách ra script khác để tránh sửa logic cũ.
- Một số script vẫn còn import namespace không dùng đến, ví dụ vài file giữ System.Collections dù gameplay hiện tại không sử dụng coroutine.
- Object reuse hiện tại là kiểu “pre-instantiate rồi bật/tắt”, chưa phải object pool tổng quát có release/borrow API riêng.

6. Kết luận ngắn

Bộ script hiện tại đã tạo thành một khối gameplay hoàn chỉnh cho game dạng Block Blast clone: có drag/drop, preview, placement validation, line clear, scoring, high score persistence, game over, restart và hỗ trợ hiển thị portrait/WebGL. Cấu trúc hiện tại đủ rõ ràng để tiếp tục mở rộng thêm animation, combo scoring, sound, VFX hoặc save/load state nếu cần.

File này được tạo tự động dựa trên nội dung thực tế của thư mục Assets/Scripts.